

Internet stvari (IoT)
(3OER6A01)

Ulazno-izlazni uređaji
Senzori i aktuatori (1.deo)

Predavanja



Senzori i aktuatori

Senzori (ulazni uređaji) su elektronske komponente koje detektuju fizičke veličine iz okruženja (npr. temperaturu, svetlost, pritisak, pokret) i pretvaraju ih u električni signal koji mikrokontroler može da obradi.

- ▶ Primeri senzora: fotootpornik (svetlost), termistor (temperatura), DHT11 (temperatura i vlažnost), PIR senzor (pokret).

Aktuatori (izlazni uređaji) su komponente koje pretvaraju električni signal iz mikrokontrolera u fizičku akciju (svetlosnu, zvučnu, mehaničku, itd).

- ▶ Primeri aktuatora: LED dioda (svetlosna signalizacija), servo motor (mehanički pokret), piezo zvučnik (zvuk), relej (uključenje/isključenje većeg uređaja).



Klasifikacija senzora

- Prema **tipu ulaza**: **digitalni** i **analogni**. (Digitalni daju binarne vrednosti (nule i jedinice), dok analogni daju kontinualne naponske vrednosti u datom opsegu (npr. 0–5V)).
- Prema **fizičkoj veličini** koju mere: **temperatura**, **svetlost**, **prisustvo gasova**, **pritisak**, **pokret**, itd.
- Prema **načinu očitavanja** vrednosti: **pasivni** i **aktivni**. (Pasivni senzori čekaju da budu očitani (zahtevaju prozivku), a aktivni senzori samostalno šalju podatke ili signale u određenim intervalima ili kada se desi promena.)

Osnovne funkcije ulaza/izlaza

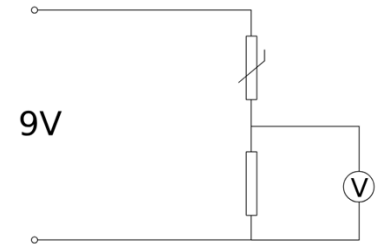
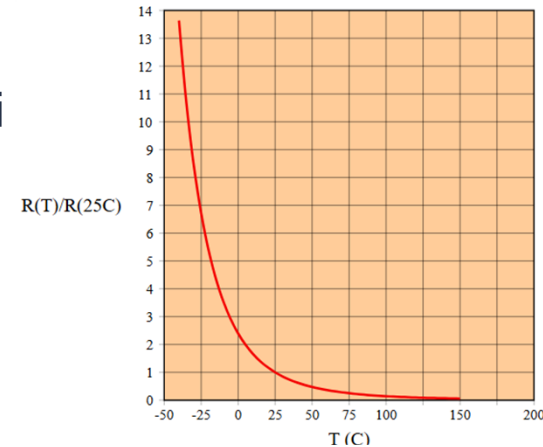
- **pinMode(pin,mode)** – režim rada – mapira *pin* na *port* i *masku*, postavlja režim u zavisnosti od *mode* na ulazni (INPUT), izlazni (OUTPUT) ili ulazni sa pull-up otpornikom (INPUT_PULLUP), tj. vrednosti u **DDRx** i **PORTx** registrima.
- **digitalRead(pin)** – digitalni ulaz – mapira *pin* na *port* i *masku*, čita odgovarajući **PINx**, izdvaja bit koji odgovara traženom pinu i vraća HIGH (1) ili LOW (0), sve preko 3V ($0.6 \times V_{cc}$) smatra se HIGH, a sve niže od 1.5V ($0.3 \times V_{cc}$) LOW (od 1.5-3V je nedefinisano stanje), zgodna ali spora ($\sim 4\mu s$).
- **digitalWrite(pin,value)** – digitalni izlaz – *pin* se mapira na *port* i *masku*, odgovarajući bit u **PORTx** registru se postavlja na *value*, pre poziva mora se postaviti pinMode(pin, OUTPUT), zgodna ali spora ($\sim 4\mu s$).

Osnovne funkcije ulaza/izlaza

- **analogRead(pin)** – **analogni ulaz (A/D)** – analogni napon na zadatom analognom pinu (A0-A15) deli referentnim naponom i prevodi u 10-bitni broj ($ADC = (V_{in}/V_{ref}) \times 1023$) u trajanju $\sim 100\mu s$, referentni napon se može promeniti pozivom funkcije **analogReference()**.
 - ▷ **analogReference(mode)** – mode: DEFAULT (V_{cc} , obično 5V), INTERNAL1V1 (1.1V), INTERNAL2V56 (2.56V), EXTERNAL (pin AREF)
- **analogWrite(pin,value)** – **pseudo-analogni izlaz (PWM)** – *pin* mora biti PWM kompatibilan (D2-D13, D44-D46), *value* u opsegu [0..255] određuje širinu impulsa (duty cycle), 255 je 100% HIGH, 127 je 50% HIGH, trajanje ciklusa zavisi od tajmera (tipično $\sim 1ms$).

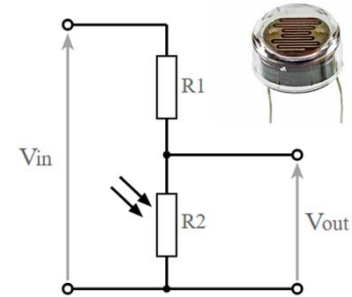
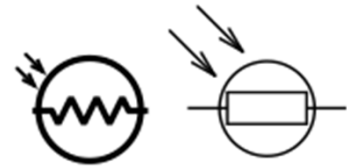
Analogni senzori – termistor

- Termistor (termo rezistor) je **pasivni analogni** senzor koji meri **temperaturu**.
- Termistor je otpornik čiji se električni otpor menja sa temperaturom.
- Može biti negativnim (**NTC**) i pozitivnim (**PTC**) temperaturnim koeficijentom.
- Karakteristika je nelinearna. Zato treba izvršiti kalibraciju (napraviti tabelu parova otpor-temperatura).
- Merenje obavljati u naponskom razdelniku, na otporniku stabilne vrednosti.

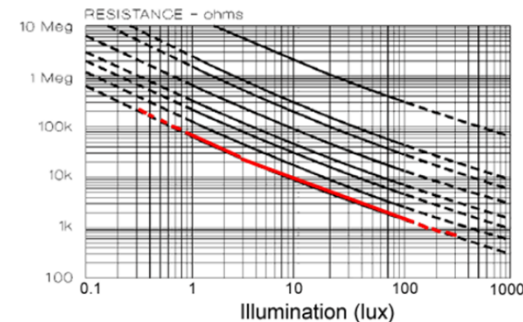


Analogni senzor – fotootpornik

- Fotootpornik (*photoresistor* ili *light dependent resistor* - LDR) je **pasivni analogni** senzor koji meri **intenzitet svetlosti**.
- Fotootpornik je otpornik čiji se električni otpor smanjuje sa povećanjem intenziteta svetlosti.
- Karakteristika je nelinearna (na grafikonu je log-log zavisnost).
- Osetljivost zavisi od talasne dužine svetlosti.



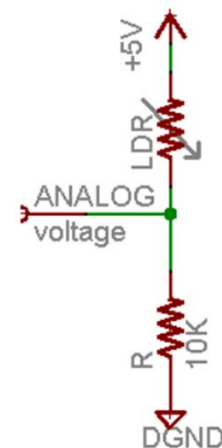
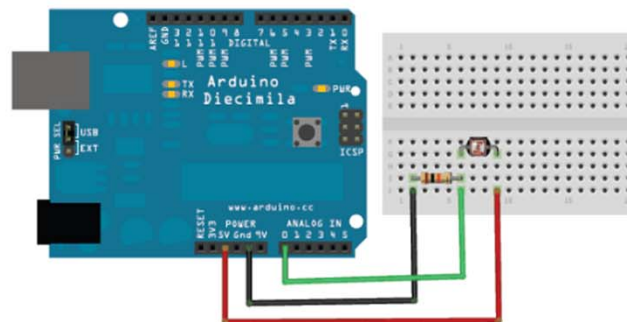
$$V_{out} = V_{in} \left(\frac{R_2}{R_1 + R_2} \right)$$



Primer fotootpornika

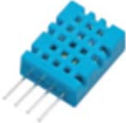
Ambient light like...	Ambient light (lux)	Photocell resistance (Ω)	LDR + R (Ω)	Current thru LDR +R	Voltage across R
Dim hallway	0.1 lux	600K Ω	610 K Ω	0.008 mA	0.1 V
Moonlit night	1 lux	70 K Ω	80 K Ω	0.07 mA	0.6 V
Dark room	10 lux	10 K Ω	20 K Ω	0.25 mA	2.5 V
Dark overcast day / Bright room	100 lux	1.5 K Ω	11.5 K Ω	0.43 mA	4.3 V
Overcast day	1000 lux	300 Ω	10.03 K Ω	0.5 mA	5V

CdS (Cadmium-Sulfide) fotootpornik PDV-P8001

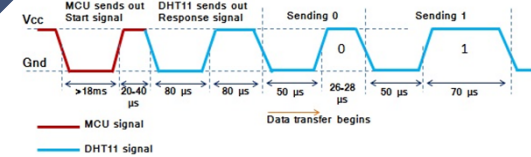


Digitalni senzor – DHT11/DHT22

- DHT11/DHT22 su **pasivni digitalni** senzori koji mere **vlagu i temperaturu**.
- DHT22 je u odnosu na DHT11 precizniji i sa širim radnim opsegom, ali je veći i sporiji (niža frekvencija očitavanja).
- Očitavanje se vrši preko jednog pina.
- Senzor šalje 5B (5×8b) podataka.

Image		
Part Number	DHT11	DHT22
Price	\$1 to \$5	\$4 to \$10
Sampling period	1 second	2 seconds
Current supply	0.5 ~ 2.5 mA	1 ~ 1.5 mA
Operating voltage	3 ~ 5.5 V	3 ~ 6 V
Temperature range	0 ~ 50 °C (+/- 2 °C)	-40 ~ 80 °C (+/- 0.5 °C)
Humidity range	20 ~ 90% (+/- 5%)	0 ~ 100% (+/- 2%)
Body size	15.5mm x 12mm x 5.5mm	15.1mm x 25mm x 7.7mm
Resolution	Humidity: 1% Temperature: 1°C	Humidity: 0.1% Temperature: 0.1°C

Digitalni senzor – DHT11/DHT22



Inicijalizacija

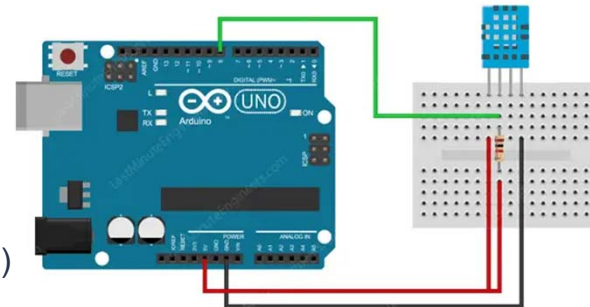
- Mikrokontroler postavlja odgovarajući pin u izlazni režim sa vrednošću **LOW** (~18ms), čime budi senzor i zahteva podatak. Zatim se prebacuje pin u ulazni režim i čeka odgovor senzora (~20-40µs).

Odziv senzora

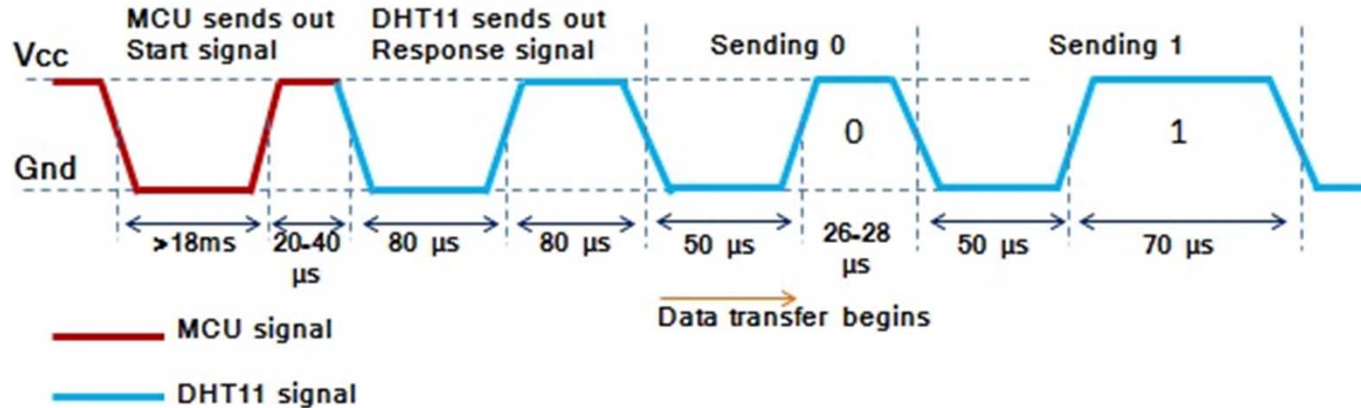
- Senzor postavlja pin na **LOW** (~80µs), pa na **HIGH** (~80µs), čime potvrđuje da je spreman da pošalje podatke.

Slanje podataka

- Senzor šalje 40 bitova (5B) u sledećem formatu:
 - 8b – vlažnost (celobrojni deo)
 - 8b – vlažnost (razlomljeni deo) (uvek 0 za DHT11)
 - 8b – temperatura (celobrojni deo)
 - 8b – temperatura (razlomljeni deo) (uvek 0 za DHT11)
 - 8b – kontrolna suma (suma prva 4B)



Digitalni senzor – DHT11/DHT22



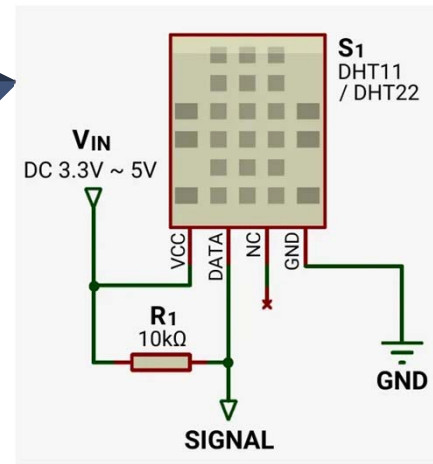
```
#include <DHT.h>
```

```
DHT dht(2, DHT11); // na (digitalnom) pinu 2 je priključen senzor tipa DHT11
```

```
dht.begin(); // uključuje pull-up
```

```
float t = dht.readTemperature();
```

```
float h = dht.readHumidity();
```



Kodiranje:

- Svaki bit počinje niskom vrednošću ($\sim 50\mu\text{s}$)
- 0 – visoki naponski nivo traje $26-28\mu\text{s}$
- 1 – visoki naponski nivo traje $70\mu\text{s}$
- 0 i 1 se razlikuju na osnovu trajanja visokog nivoa

DHT::readTemperature()

```
float DHT::readTemperature(bool force) {  
    float f = NAN;
```

```
    if (read(force)) {  
        switch (_type) {
```

```
        case DHT11:  
            f = data[2];  
            break;
```

```
        case DHT22:
```

```
            f = ((word)(data[2] & 0x7F)) << 8 | data[3];
```

```
            f *= 0.1;
```

```
            if (data[2] & 0x80) {
```

```
                f *= -1;
```

```
            }
```

```
            break;
```

```
        }
```

```
    }
```

```
    return f;
```

```
}
```

DHT11 sadrži temperaturu samo u višem bajtu (niži je uvek 0).

DHT22 predstavlja temperaturu kao znak (15-ti bit) i apsolutnu vrednost (14..0) izraženu u desetim delovima stepena.

DHT::readHumidity()

```
float DHT::readHumidity(bool force) {  
    float f = NAN;  
    if (read(force)) {  
        switch (_type) {  
            case DHT11:  
                f = data[0];  
                break;  
            case DHT22:  
                f = ((word)data[0]) << 8 | data[1];  
                f *= 0.1;  
                break;  
        }  
    }  
    return f;  
}
```

```
bool DHT::read(bool force) {  
    // Check if sensor was read less than two seconds ago  
    // and return early to use last reading.  
    uint32_t currenttime = millis();  
    if (!force && ((currenttime - _lastreadtime) < MIN_INTERVAL)) {  
        return _lastresult; // return last correct measurement  
    }  
    _lastreadtime = currenttime;  
  
    // Reset 40 bits of received data to zero.  
    data[0] = data[1] = data[2] = data[3] = data[4] = 0;
```

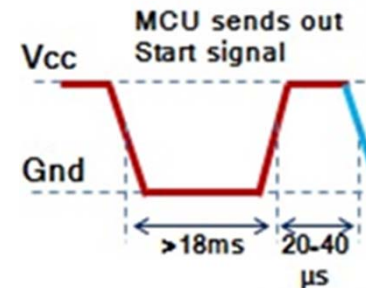
DHT::read()

```
// Go into high impedance state to let pull-up raise data line level and  
// start the reading process.
```

```
pinMode(_pin, INPUT_PULLUP);  
delay(1);
```

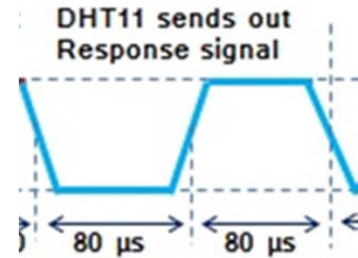
```
// First set data line low for a period according to sensor type
```

```
pinMode(_pin, OUTPUT);  
digitalWrite(_pin, LOW);  
switch (_type) {  
case DHT22:  
    delayMicroseconds(1100); // data sheet says "at least 1ms"  
    break;  
case DHT11:  
default:  
    delay(20); // data sheet says at least 18ms, 20ms just to be safe  
    break;  
}
```



DHT::read()

```
uint32_t cycles[80];  
{  
    // End the start signal by setting data line high for 40 microseconds.  
    pinMode(_pin, INPUT_PULLUP);  
  
    // Delay a moment to let sensor pull data line low.  
    delayMicroseconds(pullTime);  
  
    // Now start reading the data line to get the value from the DHT sensor.  
    // Turn off interrupts temporarily because the next sections  
    // are timing critical and we don't want any interruptions.  
    InterruptLock lock;  
  
    // First expect a low signal for ~80 microseconds followed by a high signal for ~80 microseconds again.  
    if (expectPulse(LOW) == TIMEOUT) {  
        _lastresult = false;  
        return _lastresult;  
    }  
}
```



DHT::read()

```
if (expectPulse(HIGH) == TIMEOUT) {  
    _lastresult = false;  
    return _lastresult;  
}  
// Now read the 40 bits sent by the sensor. Each bit is sent as a 50  
// microsecond low pulse followed by a variable length high pulse. If the  
// high pulse is ~28 microseconds then it's a 0 and if it's ~70 microseconds  
// then it's a 1. We measure the cycle count of the initial 50us low pulse  
// and use that to compare to the cycle count of the high pulse to determine  
// if the bit is a 0 (high state cycle count < low state cycle count), or a  
// 1 (high state cycle count > low state cycle count). Note that for speed  
// all the pulses are read into a array and then examined in a later step.  
for (int i = 0; i < 80; i += 2) {  
    cycles[i] = expectPulse(LOW);  
    cycles[i + 1] = expectPulse(HIGH);  
}  
} // Timing critical code is now complete.
```


DHT::read()

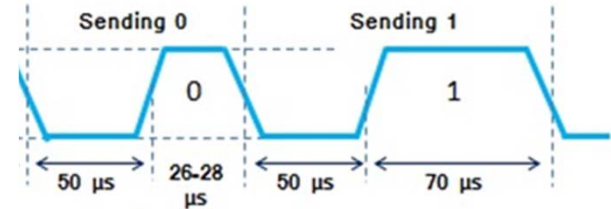
```
// Inspect pulses and determine which ones are 0 (high state cycle count < low  
// state cycle count), or 1 (high state cycle count > low state cycle count).
```

```
for (int i = 0; i < 40; ++i) {  
    uint32_t lowCycles = cycles[2 * i];  
    uint32_t highCycles = cycles[2 * i + 1];  
    if ((lowCycles == TIMEOUT) || (highCycles == TIMEOUT)) {  
        _lastresult = false;  
        return _lastresult;  
    }  
    data[i / 8] <<= 1;
```

```
// Now compare the low and high cycle times to see if the bit is a 0 or 1.
```

```
if (highCycles > lowCycles) {  
    data[i / 8] |= 1; // High cycles are greater than 50us low cycle count, must be a 1.
```

```
}  
// Else high cycles are less than (or equal to, a weird case) the 50us low  
// cycle count so this must be a zero. Nothing needs to be changed in the  
// stored data.
```



DHT::read() / DHT::expectPulse()

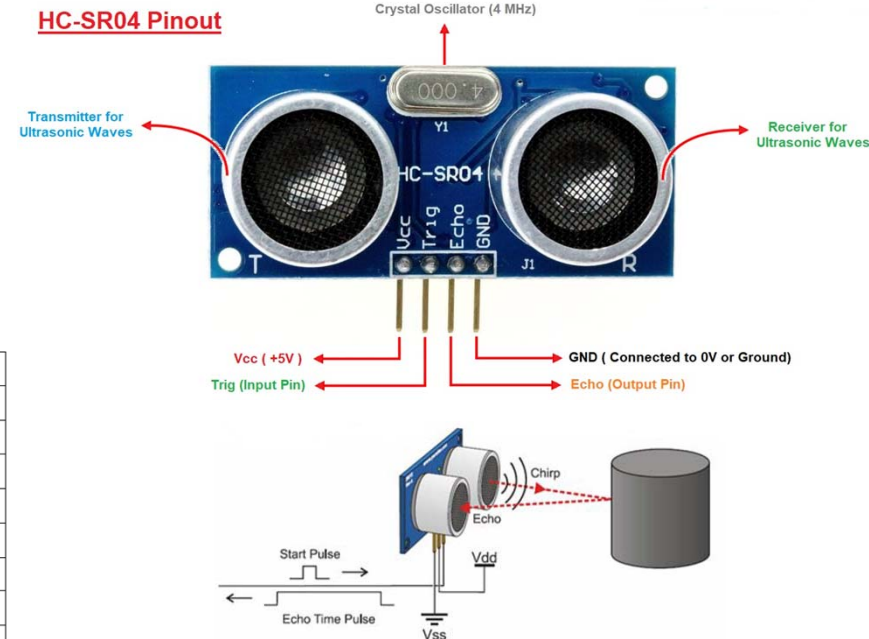
// Check we read 40 bits and that the checksum matches.

```
if (data[4] == ((data[0] + data[1] + data[2] + data[3]) & 0xFF)) {  
    _lastresult = true;  
    return _lastresult;  
} else {  
    _lastresult = false;  
    return _lastresult;  
}  
}  
  
uint32_t DHT::expectPulse(bool level)  
{  
    uint16_t count = 0;  
    while (digitalRead(_pin) == level) {  
        if (count++ >= _maxcycles) {  
            return TIMEOUT; // Exceeded timeout, fail.  
        }  
    }  
    return count;  
}
```

Digitalni senzor – HC-SR04

- HC-SR04 je ultrazvučni senzor rastojanja.
- Domet: 2cm do 4m.
- Trajanje *echo* impulsa:
 - 117us do 23ms kada postoji prepreka
 - 38ms kada nema prepreke

Electrical Parameters	Value
Operating Voltage	3.3Vdc ~ 5Vdc
Quiescent Current	<2mA
Operating Current	15mA
Operating Frequency	40KHz
Operating Range & Accuracy	2cm ~ 400cm (1in ~ 13ft) ± 3mm
Sensitivity	-65dB min
Sound Pressure	112dB
Effective Angle	15°
Connector	4-pins header with 2.54mm pitch
Dimension	45mm x 20mm x 15mm
Weight	9g



Digitalni senzor – HC-SR04

Okidanje

- MCU šalje **HIGH** impuls u trajanju $\geq 10\mu\text{s}$ na **TRIG** pin.

Emitovanje

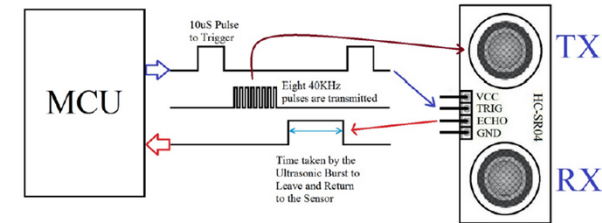
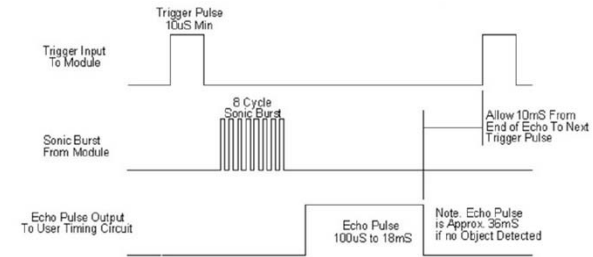
- Senzor šalje 8 ultrazvučnih talasa frekvencije 40KHz

Odgovor

- Senzor postavlja **ECHO** pin na **HIGH** kada se vrati eho poslatih talasa i zadržava ga onoliko koliko je talasu trebalo da dosegne cilj i vrati se.

Računanje rasrojanja

- Rastojanje = $(\text{trajanje} * \text{brzina_zvuka}) / 2$.
- Brzina zvuka: 343 m/s



long duration = **pulseIn(echoPin, HIGH);**
float distance = duration * 0.0343 / 2;
// rastojanje u cm

Digitalni senzor – HC-SR04

```
int trig=9;
int echo=8;
int duration;
float distance;
float meter;
void setup()
{
  Serial.begin(9600);

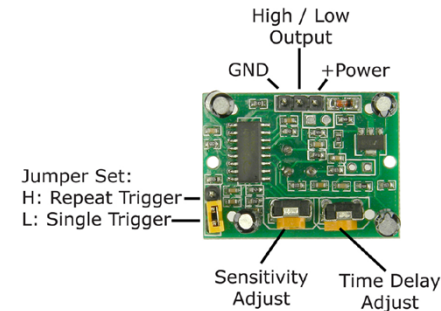
  pinMode(trig, OUTPUT);
  digitalWrite(trig, LOW);
  delayMicroseconds(2);
  pinMode(echo, INPUT);
  delay(6000);

  Serial.println("Distance:");
}

void loop()
{
  digitalWrite(trig, HIGH);
  delayMicroseconds(10);
  digitalWrite(trig, LOW);
  duration = pulseIn(echo, HIGH); // trajanje u us, preciznost (1-2us)
  if(duration >= 38000) // impuls traje maksimalno 38us, kada nema prepreke
  {
    Serial.print("Out range");
  }
  else
  {
    distance = duration / 58;
    Serial.print(distance); Serial.print("cm");
    meter=distance/100; Serial.print("\t");
    Serial.print(meter); Serial.println("m"); } delay(1000);
}
```

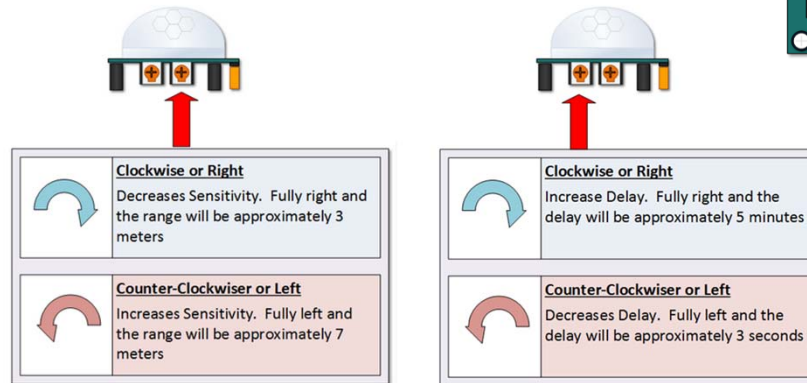
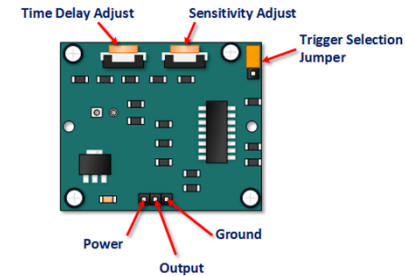
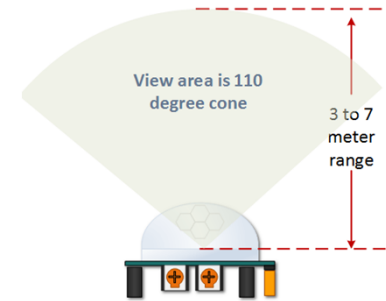
Digitalni senzor – HC-SR501 PIR

- HC-SR501 je pasivni infra-red (PIR) senzor pokreta, podesivog dometa (3-7m).
- Senzor detektuje promenu u infra-crvenom delu spektra, tj. detektuje promenu toplote, i to interpretira kao pokret.
- Zahteva minut da se inicijalizuje i za to vreme ne može se koristiti za detekciju pokreta, jer može emitovati lažne signale.
- Može se podesiti:
 - **Trajanje signala** („detektovan pokret“) od **3s** do **5min** putem potencijometra
 - **Osetljivost** (domet) od **3m** do **7m** putem potencijometra
 - **Režim rada** (**pojedinačno** ili **ponovljeno okidanje**) putem džampera (kratkospojnika)



Digitalni senzor – HC-SR501 PIR

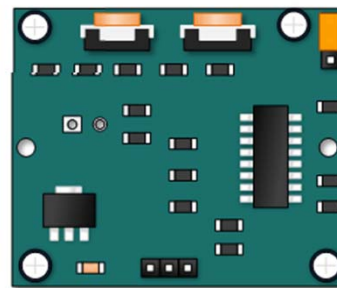
- Otkriva pokret u kupi sa vršnim uglom od **110°**.
- Napaja se sa **5-20V** preko **Power** pina.
- Output** pin je na **LOW** kada nije detektovan pokret, a na **HIGH** kada jeste. **HIGH** je **3.3V**! Zahvaljujući dovoljno velikoj margini kod Arduina, ipak se može detektovati direktnim povezivanjem na ulazni pin.
- Potenciometar **Sensitivity** u krajnje desnom položaju – domet **3m**, u krajnjem levom – **7m**.
- Potenciometar **Delay** u krajnje desnom položaju – **5min**, u krajnje desnom – **3s**.



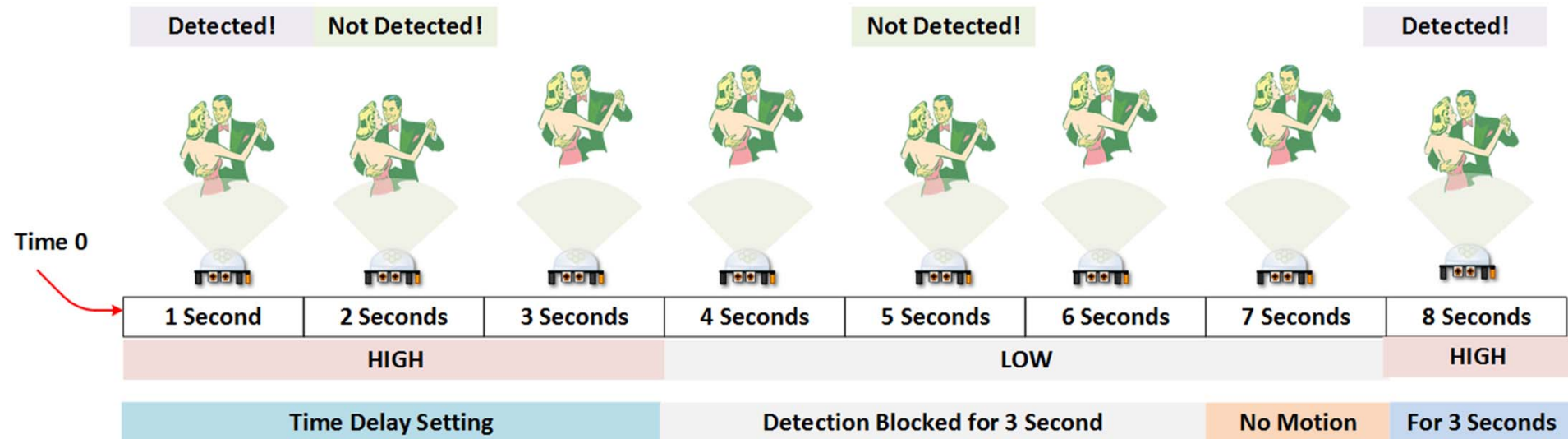
Digitalni senzor – HC-SR501 PIR

- Kada se desi detekcija, **Output** pin ide na **HIGH** i ostaje u tom stanju u trajanju podešenom **Delay** potencijometrom (**3s–5min**).
- Nakon toga, **Output** pin ide na **LOW** i ostaje tako **~3s**, kada ne može da detektuje nove pokrete (vreme oporavka senzora – refractory period).
- U **Single Trigger** modu, događaj okida tajmer i do isteka vremena **Delay+3s** ne mogu se detektovati novi pokreti. Jednokratni impuls detektor. Fiksno vreme trajanja događaja.
- U **Repeatable Trigger** modu, tajmer se resetuje svaki put kada se detektuje pokret dok je **Output** na **HIGH**. Promenljivo vreme trajanja događaja.
- Trajanje **Delay** se podešava u skladu sa primenom. Kratko (5s) – brojač prolazaka, signalni zvuk, dugo (2min) – osvetljenje stepeništa, alarm.
- Osetljivost (**Sensitivity**) takođe zavisi od primene. Kratak domet (3m) – male sobe, precizne zone, duži domet (7m) – hodnici, otvoren prostor.

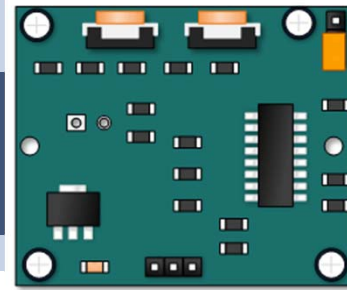
Single Trigger mod



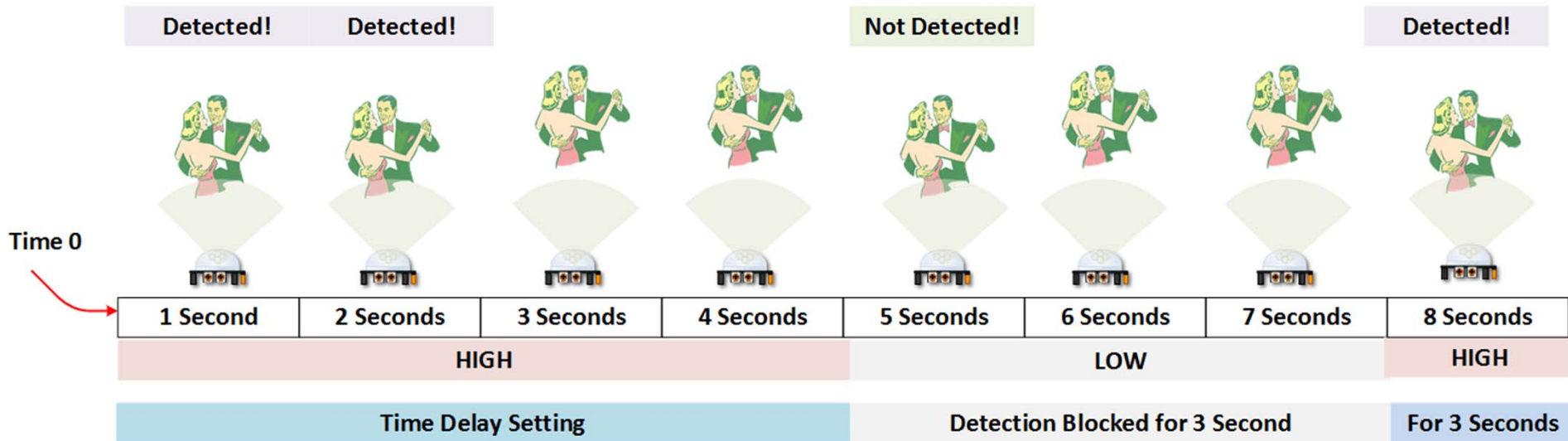
Single Trigger Mode – Time Delay is started immediately upon detecting motion. Continued detection is blocked



Repeatable Trigger mod



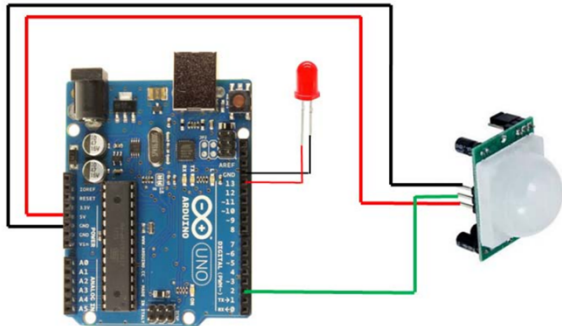
Repeatable Trigger Mode –
Time Delay is re-started every
time motion is detected.



Digitalni senzor – HC-SR501 PIR

```
int led = 13; // the pin that the LED is attached to
int sensor = 2; // the pin that the sensor is attached to
int state = LOW; // by default, no motion detected
int val = 0; // variable to store the sensor status (value)
```

```
void setup()
{
  pinMode(led, OUTPUT); // initialize LED as an output
  pinMode(sensor, INPUT); // initialize sensor as an input
  Serial.begin(9600); // initialize serial
}
```

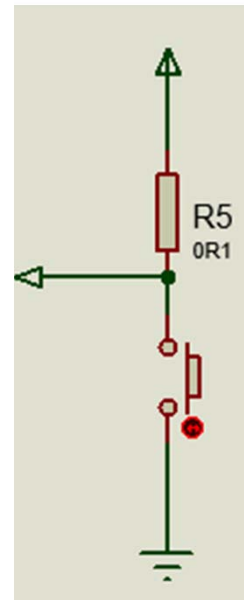


```
void loop()
{
  val = digitalRead(sensor); // read sensor value
  if (val == HIGH) { // check if the sensor is HIGH
    digitalWrite(led, HIGH); // turn LED ON
    delay(100); // delay 100ms
    if (state == LOW) {
      Serial.println("Motion detected!");
      state = HIGH; // update variable state to HIGH
    }
  }
  else {
    digitalWrite(led, LOW); // turn LED OFF
    delay(200); // delay 200ms
    if (state == HIGH){
      Serial.println("Motion stopped!");
      state = LOW; // update variable state to LOW
    }
  }
}
```

Taster

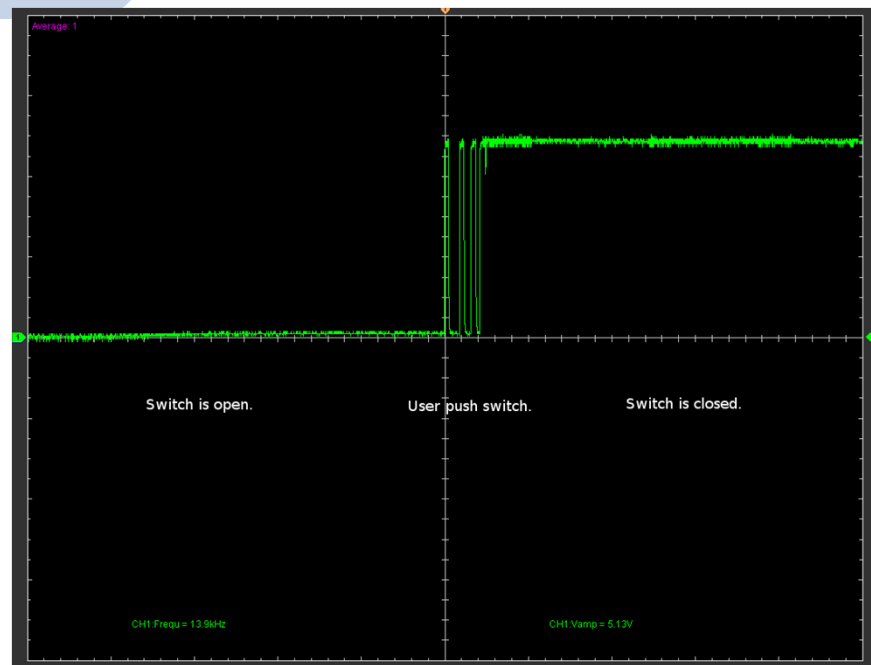


- Taster je **digitalni ulazni uređaj** koji omogućava korisniku da pošalje signal mikrokontroleru.
- U osnovi je to **prekidač** koji može biti u dva stanja: **zatvoren** (pritisnut) ili **otvoren** (otpušten).
- Taster se najčešće koristi za pokretanje akcija, promenu stanja ili interakciju sa korisnikom.
- Taster se povezuje između digitalnog pina i uzemljenja (GND).
- Potreban je **pull-up otpornik** (eksterni ili interni) kako bi pin imao definisano stanje kada je taster otpušten.
- Kada je **otpušten**, ulaz je na **HIGH** (preko pull-up otpornika), a kada je **pritisnut** na **LOW** (povezan na GND).



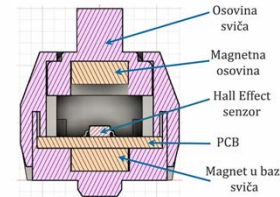
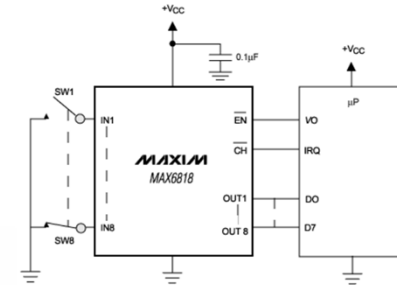
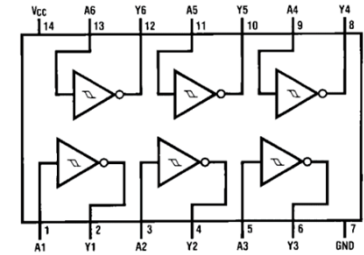
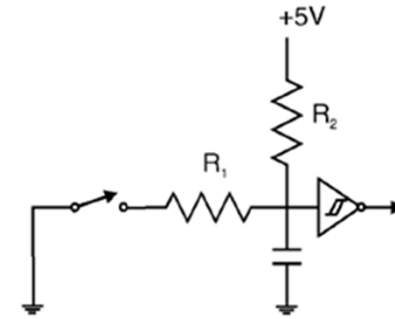
Treperenje tastera

- Kada se pritisne ili otpusti taster, on „treperi“ (između stanja „otvoreno“ i „zatvoreno“) sve dok konačne ne ostane u odgovarajućem stanju.
- Period treperenja zavisi od konstrukcije.
- Kvalitetniji tasteri trepere od 1ms do 5ms, a manje kvalitetni od 20ms do čak 100ms.
- Problem se može rešiti hardverski ili softverski.



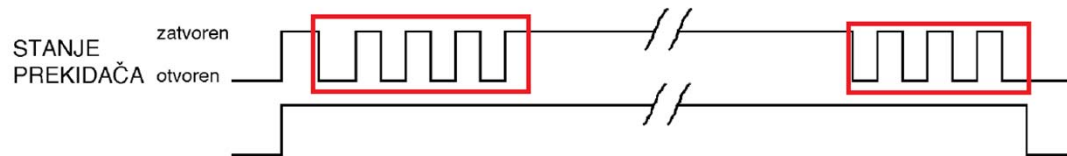
Hardverske tehnike za eliminaciju treperenja tastera

- RC niskopropusni filter (npr. $R=10k\Omega$ $C=0.47\mu F$)
- Schmitt trigger (npr. 74HC14)
- Specijalizovana integrisana kola (npr. MAX6816, MAX6817, MAX6818)
- Mehanički debounce tasteri ili enkoderi (Treperenje je jako smanjeno, obično na 5ms, kod najkvalitetnijih na 1ms. Kod tastera sa Hall-ovim efektom nema uopšte treperenja jer nema fizičkog kontakta.)



Softverske tehnike za eliminaciju treperenja tastera

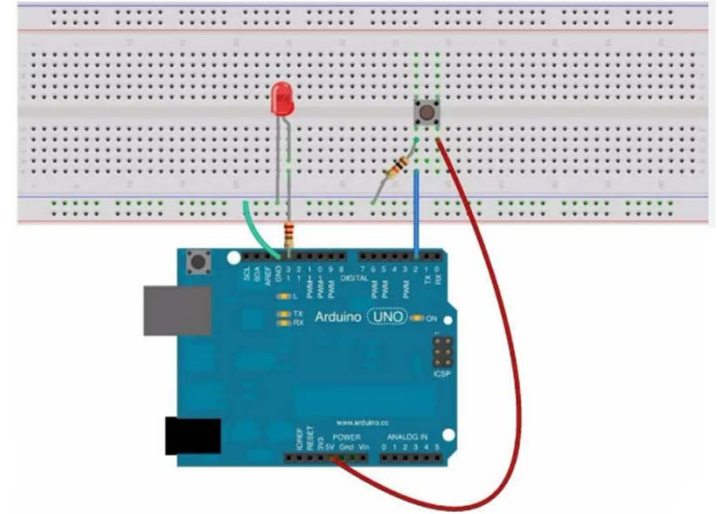
- U potpunosti se obavlja programski.
- Aktivira se brojač, čiji period odgovara trajanju treperenja.
- Kada se detektuje promena stanja tastera, stanje (tj. kod tastera) se privremeno memoriše.
- Ako je po isteku brojača očitano stanje isto kao početno, smatra se da je taster u tom položaju, a kod tastera se predaje rutini za obradu.
- Može se uvesti eliminacija samo zatvaranja ili otvaranja prekidača (npr. čim prvi put pređe u zatvoreno stanje, smatra se da je zatvoren, ali se pri prelasku u otvoreno stanje odbrojava, pa ako nije otvoren nakon npr. 10ms, smatra se da je zatvoren).



Primer

```
const int buttonPin = 2;    // Button connected to digital pin 2
const int ledPin = 13;      // LED connected to digital pin 13 (optional)
int buttonState;            // the current reading from the input pin
int lastButtonState = LOW;  // the previous reading from the input pin
long lastDebounceTime = 0;  // the last time the output pin was toggled
long debounceDelay = 50;   // the debounce time in milliseconds
```

```
void setup() {
  pinMode(buttonPin, INPUT_PULLUP);
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600); // optional for debugging
}
```



Primer

```
void loop() {  
  int reading = digitalRead(buttonPin);  
  
  if (reading != lastButtonState) {  
    lastDebounceTime = millis();  
  }  
  if ((millis() - lastDebounceTime) > debounceDelay) {  
    if (reading != buttonState) {  
      buttonState = reading;  
      if (buttonState == HIGH) {  
        digitalWrite(ledPin, !digitalRead(ledPin));  
        Serial.println("Button Pressed");  
      }  
    }  
  }  
  lastButtonState = reading;  
}
```

// Check to see if you just pressed the button
// reset the debouncing timer

// if the button state has changed

// only toggle the LED if the new button state is HIGH
// toggle the LED
// optional for debugging

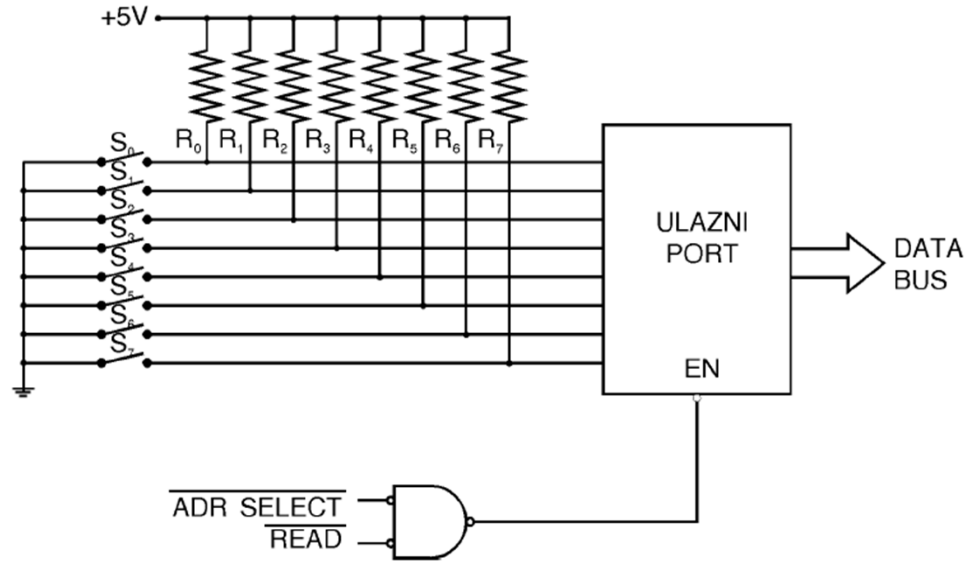
// save the current state as the last state for next time

Tastatura

- Grupa označenih tastera uređenih na određeni način čini **tastaturu**.
- Tastatura se u praksi kombinuje sa nekom vizuelnom izlaznom jedinicom – **displejem**.
- **Tastatura + displej** čine jedinicu koju nazivamo – **terminal**.
- Zavisno od broja tastera, najčešće se koriste sledeća dva tipa povezivanja:
 - **Nemultipleksirani** interfejs – pogodan za tastature sa malim brojem tastera ili za očitavanje postavljene konfiguracije (DIP prekidači ili kratkospajajući)
 - **Multipleksirani** interfejs – pogodan za tastature sa većim brojem tastera

Nemultipleksirani interfejs

- Jednostavno rešenje, primenljivo za mali broj prekidača/tastera.
- Za svaki prekidač/taster potreban je poseban ulazni pin.



Multiplesirani interfejs

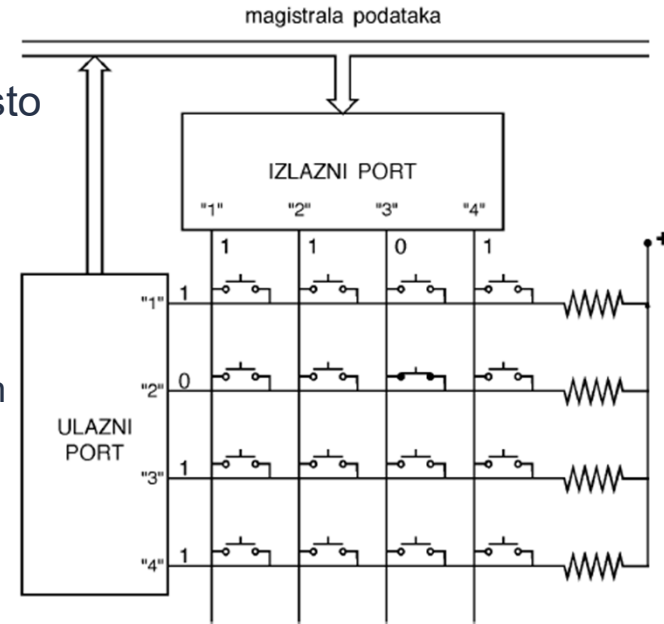
Potreban je manji broj U/I bitova.

Za 16 tastera treba 4-bitni ulazni i 4-bitni izlazni port (umesto 16-bitnog ulaznog porta kod nemultipleksiranog)

Tehnika „**šetajuće nule**“

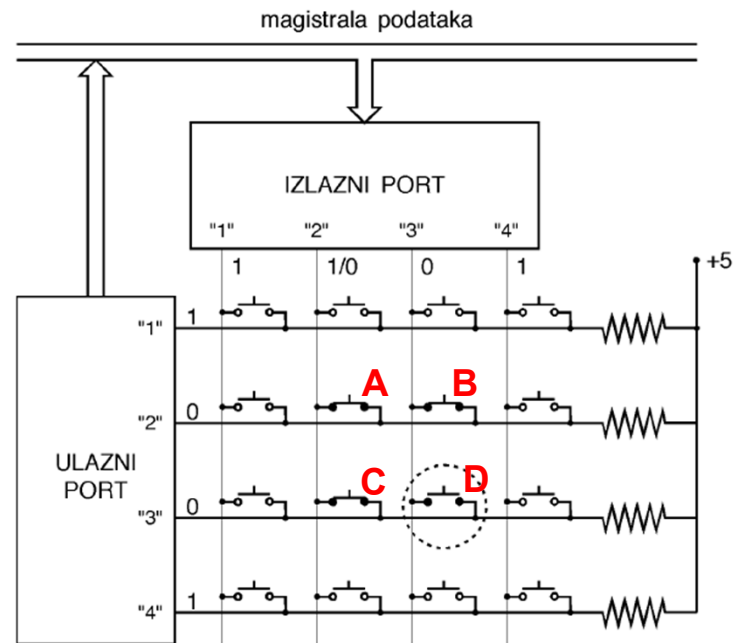
- ▶ jedan pin izlaznog porta postavlja se na 0, a svi ostali na 1 (selektuje se jedna kolona)
- ▶ pritiskom tastera u selektovanoj koloni, postavlja 0 na jednom od pinova ulaznog porta
- ▶ vrednost na izlaznom portu se kružno pomera i time se omogućuje očitavanje tastature kolona po kolona

Korišćenjem programabilnog tajmera, može se generisati zahtev za prekidom kojim se analizira jedna kolona tastature; ako postoji 0 na ulaznom portu, pritisnuti taster se dekodira na osnovu vrednosti na ulaznom i izlaznom portu



„Ghost keys“ efekat

- Problem pritiska više tastera istovremeno kod matrične tastature.
- Selektovana je kolona 3. Pritisnut taster B se regularno očitava.
- Pritisnut A dovodi izlaz 2 na 0 (port 2 ga napaja, ali ga 3 odvodi na 0).
- Pritisnut C dovodi 0 na ulaz 3. Obzirom da je selektovana kolona 2, smatra se da je **pritisnut i D**.
- Može se rešiti vezivanjem **diode na red sa svakim tasterom**.



Ključni pojmovi

- Senzori i aktuatori
- Klasifikacija senzora
- Osnovne funkcije ulaza/izlaza
- Analogni senzori
 - ▷ Termistor
 - ▷ Fotootpornik
- Digitalni senzori
 - ▷ DHT11/22
 - ▷ HC-SR04
 - ▷ HC-SR501
- Tasteri/Treperenje
- Tastatura (Nemultipleksirana i multipleksirana)

PITANJA

